

Afonin Oleksandr

A Deeper Look into Game Development Tools

Bachelor's thesis

Bachelor of Engineering

Information Technology

2025



South-Eastern Finland
University of Applied Sciences

Degree title	Bachelor of Engineering
Author	Oleksandr Afonin
Thesis title	Game Development Tools Research
Year	2025
Pages	26
Supervisor	Miika Reijonen

ABSTRACT

Game development is a complex, multidisciplinary process that involves collaboration between people and a variety of tools, frameworks and creative disciplines. Ever since the early 90s, game development as a form of art has been evolving in giant leaps. At the core of this practice lies the critical interaction between game design and programmers, one of the most notoriously difficult parts of the process yet often underestimated by those outside the industry.

This thesis investigated the interface of game art and programming through the lens of the level editor, one of the most fundamental development tools. The objective was to deepen the understanding of game development workflows while providing a customizable tool to support the creation and testing of game levels.

As a practical component of this study, a custom level editor was developed using C++ to gain firsthand insight into the challenges and solutions involved in the process. This study explored how level editors can serve as a link between artistic vision and technical implementation, streamlining workflows and enhancing collaboration between departments.

The thesis highlights the general importance of internal tools in game production pipelines, defining the problems they address and the efficiencies they create. By analysing both the development process and the role level editors play in real-world scenarios, this thesis offers a focused contribution to the understanding of game development tools and their significance in the industry.

Keywords: game development, level editor, C++, game tools, internal tools, artist-programmer collaboration, game production pipeline

CONTENTS

1	INTRODUCTION	6
1.1	Personal Motivation and Project Goals	7
2	BACKGROUND AND MOTIVATION.....	8
2.1	Existing Solutions in 2D Game Development.....	8
2.2	Technologies Used.....	10
3	REQUEREMENTS AND PLANNING	11
4	TECHNOLOGY STACK	13
4.1	Frontend / UI	13
4.2	Backend / Core Logic.....	14
4.3	Database	14
5	IMPLEMENTATION	15
5.1	Core Features	15
5.2	Grid System	15
5.3	Camera Work	16
5.4	Layers	16
5.5	Tile Selection and Placement.....	16
5.6	Select Tool	17
5.7	Tileset Categorization.....	18
5.8	Collision System.....	19
5.9	Saving and Loading Levels	19
6	CHALLENGES AND SOLUTIONS	20
6.1	UI Scaling	20

6.2	Serialization.....	21
7	FUTURE IMPROVEMENTS.....	22
8	CONCLUSION	23
	REFERENCES.....	26
	LIST OF FIGURES	27

1 INTRODUCTION

Game development is a collaborative effort that involves a range of disciplines working together toward a unified goal. Among the most critical relationships in this process is the interaction between programmers and artists. While programmers are responsible for the underlying mechanics and functionality, artists bring the game world to life with visuals, animations, and environments. However, the workflow withing a project often reveals a rather large gap, be it understanding each other's roadblocks, creative differences or simply technical limitations. Artists frequently rely on programmers to implement and test visual elements, which can slow down production, lead to miscommunication, and hinder the creative workflow. This reliance can become a bottleneck, especially in smaller teams or during early stages of development where rapid iteration is essential.

The specific problem addressed in this study revolves around the absence of tools for artists to test and iterate on their assets without needing constant involvement from a programmer. When an artist creates visual assets such as tilesets or environment sprites, they often struggle to see how those assets will appear and function within the actual game environment. This does not only delay progress but also limits artistic control. What is needed is a tool, essentially a separate product that empowers artists to visualize and organize their work in context, ensuring that the assets they produce are both usable and game-ready, while giving the artist some idea of how their work will appear in the end product.

In order to address this issue, a standard 2D level editor was developed as part of this study. The tool includes all the expected features of a professional-level editor: tile placement, undo/redo functionality, tile deletion, level saving, layer support, tileset management, categorization of tiles, and the ability to incorporate background music, albeit on a very rudimentary level. While the editor does not

introduce any groundbreaking new concepts, it provides a reliable and accessible environment in which artists can independently build and test level layouts. This allows for quicker iteration, greater creative freedom, and smoother collaboration between disciplines.

The motivation for this development lies in the fact that level editors have become a core component of modern game development workflows. They enable non-programmers, especially artists and designers, to contribute meaningfully to the game without being blocked by technical barriers. By creating a practical, fully functioning level editor, this thesis aims to replicate and explore a common industry-standard solution, demonstrating not only its technical feasibility but also its real-world value in improving production efficiency and interdisciplinary collaboration.

1.1 Personal Motivation and Project Goals

I have always maintained a strong interest in the gaming industry and in video games as a form of artistic expression. Being fascinated with video games from an early age, it was only a matter of time before I would attempt to develop something of my own—initially for training and learning purposes.

2D development appeared to be the ideal starting point due to its relatively low entry level compared to 3D game development which often requires the use of more complex engines such as Unity. Another motivating factor at the time was my fascination with *Barotrauma* published in 2019 by FakeFish which features all the essential elements of a 2D sandbox experience and includes a particularly advanced level editor (see Figure 1) (FakeFish & Undertow Games n.d.). The technology stack selected for this thesis closely mirrors the stack used in *Barotrauma*'s development. Since I had already become familiar with the game's scripting environment through prior exploration, it served as both a practical reference and an inspirational benchmark for my own project.

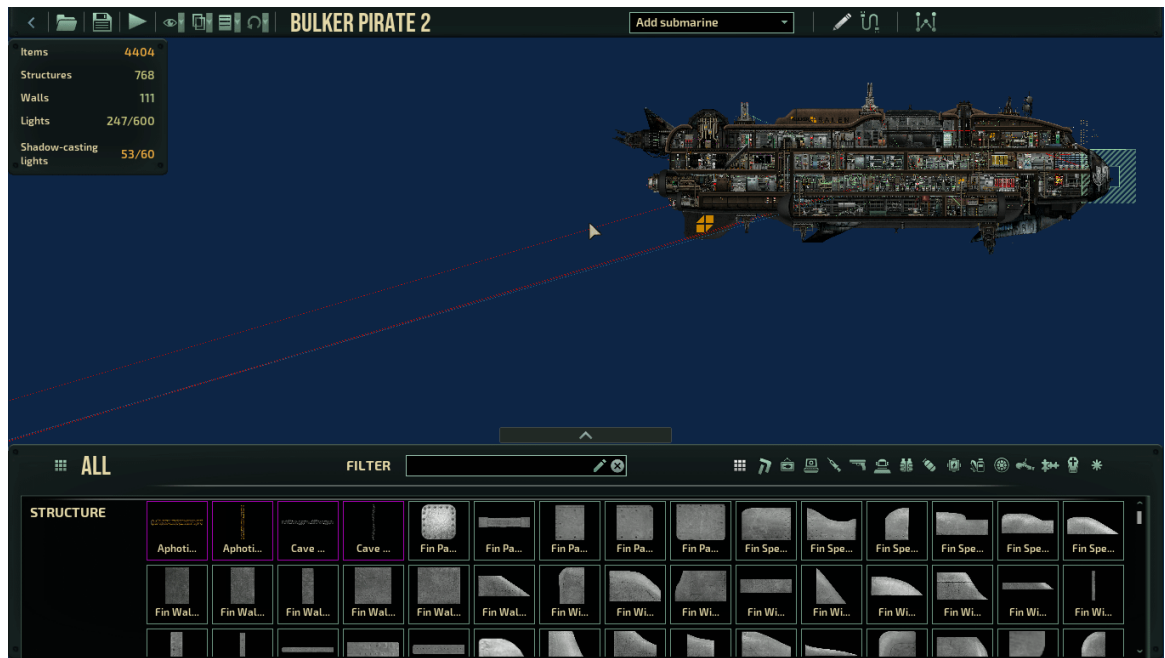


Figure 1. A screenshot from the game “Barotrauma” which features the level editor (FakeFish & Undertow Games n.d.)

2 BACKGROUND AND MOTIVATION

2.1 Existing Solutions in 2D Game Development

The field of 2D game development has matured significantly over the past two decades, with a wide range of tools and frameworks now available to developers. One of the key aspects of 2D game production is the use of level editors—tools that allow developers and artists to design game environments, place interactive objects, and structure gameplay areas visually. These tools serve not only as productivity enhancers but also as essential bridges between different disciplines, such as art and programming, allowing each part of the team to focus on their work without worrying about integration, which greatly streamlines development. (Tiled n.d.; Raylib n.d.; Unity Technologies n.d.)

A prominent example of a widely used level editor is Tiled, an open-source 2D level editor that supports both orthogonal and isometric map layouts. Tiled allows developers to create tilemaps using a drag-and-drop interface and supports features such as layers, object groups, tileset definitions, and custom properties.

It exports to various formats (such as JSON and TMX), making it easy to integrate with custom engines or game frameworks. Because of its open-source nature and flexibility, Tiled has been adopted by independent developers and professionals, and continues to serve as a reference standard in the industry. (Tiled n.d.)

Another major platform is Unity which, while primarily a 3D engine, offers extensive support for 2D game development. Unity provides built-in tools for tilemap editing, sprite layering, and animation, and supports custom editor scripting that allows teams to build their own tools within the Unity environment. While powerful, Unity's full capabilities often come with a steep learning curve and can be more complex than needed for small-scale or highly customized projects (Unity Technologies n.d.) It was briefly considered at the early stages of this study when choosing an engine, but was eventually disregarded as redundant.

Godot is another open-source game engine that has seen a surge in popularity. Its 2D engine is separate from its 3D counterpart, ensuring better performance and usability for 2D games. Godot includes its own integrated level editor, scripting support via GDScript (similar to Python), and user-friendly scene management. Because it is open-source, it is a suitable choice for smaller teams or those looking to fully customize their development pipeline (Godot Engine n.d.)

In contrast to full-fledged engines, frameworks such as LÖVE (Love2D) and Phaser (a 2D HTML5 game framework) offer more code-driven approaches to 2D game development. These platforms usually rely on external tools or custom-built editors for level design. As such, many developers in these ecosystems create their own level editors or adapt existing ones such as Tiled to suit their workflows (LÖVE Development Team n.d.; Photon Storm n.d.; Tiled n.d.).

In summary, the landscape of tools and frameworks for 2D game development is diverse. Open-source solutions, namely Tiled and Godot, provide accessible and

flexible options for teams looking to customize their tools, while platforms such as Unity offer powerful out-of-the-box features for larger projects (Tiled n.d.; Godot Engine n.d.; Unity Technologies n.d.). However, despite the availability of these tools, there is still a need for lightweight, purpose-built editors tailored to specific workflows, particularly for enabling artists to test and iterate on assets without relying on a programmer. This ongoing need, and the desire to experiment, is what motivates the creation of custom tools, as exemplified by the one developed in this study.

2.2 Technologies Used

In order to develop a 2D level editor, a combination of programming languages and tools was selected with a focus on performance, accessibility, and ease of integration. The core technologies used include C++ for the main application logic, Raylib for rendering and user input, and Lua for scripting support. Development and debugging were carried out primarily in Visual Studio Code (VS Code). A lightweight but powerful source code editor widely used in both professional and hobbyist development environment.

C++ was chosen as the primary language due to its high performance and wide use in game development. As a systems-level language, C++ offers precise control over memory and hardware resources, making it ideal for applications where efficiency is a priority. It also has a long history in the games industry, with most major engines and tools—such as Unreal Engine, CryEngine, and parts of Unity—built on or heavily reliant upon C++. For this project, C++ allowed the creation of a fast, responsive level editor capable of handling graphical rendering, input management, and file operations in a performant manner.

For the graphical rendering and game-like interface of the level editor, Raylib was used. Raylib is a simple and easy-to-use C library designed specifically for game development and graphical applications. It provides a clean API for managing windows, drawing textures, handling input devices, and playing audio. Unlike

larger frameworks or engines, Raylib does not come with the overhead of complex scene graphs or asset pipelines, which makes it ideal for a lightweight, standalone tool (Raylib n.d.). Its focus on simplicity and minimalism also made the development process more manageable and transparent, particularly when building a custom editor from scratch.

In order to allow for potential extendibility and future customization Lua is being planned to be integrated as a scripting language. Lua is a lightweight, embeddable scripting language that is commonly used in game development to give designers and artists more control without modifying core application code (Lua.org n.d.). Although Lua's role in this study is very limited, its inclusion opens up the possibility of allowing users to define behaviours, automate editor tasks, or script certain types of content placement in the future iterations of the tool.

In this study level editor was developed using Visual Studio Code (VS Code), a widely adopted code editor that supports C++, Lua, and many other languages through extensions. VS Code offers an efficient environment for writing and organizing code, along with features such as integrated debugging, Git integration, syntax highlighting, and linting languages (Microsoft n.d.). Its lightweight nature, combined with a rich extension ecosystem, made it a fitting choice for a project that required flexibility and ease of configuration across different libraries and languages.

Together, Raylib and C++ provided a solid and flexible foundation for building the level editor while keeping the development process accessible and efficient.

3 REQUIREMENTS AND PLANNING

A significant effort in this study was dedicated to the planning and design phase, as establishing a solid foundation was essential for the success of the level editor. Rather than immediately diving into coding, time was invested in carefully selecting the right tools, defining the scope, and determining the key features that

would make the editor functional, usable, and adaptable for 2D game development workflows.

The first step during planning involved selecting the core technologies that would support the development process. Given the objective of performance, simplicity, and flexibility, C++ was chosen as the main programming language, with Raylib as the graphics and input handling library. Raylib was selected because of its straightforward API and relative ease of integration into small- to medium-scale graphical applications. This choice allowed for full control over the rendering pipeline and input system while keeping the development process manageable. Visual Studio Code (VS Code) was selected as the primary development environment due to its flexibility, support for C++ development through extensions, and ease of configuration for multi-language projects. Together, these tools formed the core of the development stack.

Once the technical stack was established, the functional requirements of the editor had to be defined. The focus was on implementing a standard feature set commonly found in professional level editors to ensure that the tool would be practical and effective. Some of the fundamental features identified early in the planning phase included camera controls for navigating the level space, save and load functionality to enable persistent work across sessions, and a tileset system for managing graphical assets. These tilesets were designed to be categorized to help artists and designers easily find and organize different types of tiles, such as terrain, decorations, or interactive elements.

Special consideration was also given to user experience. Features such as undo and redo, tile placement and deletion, and multi-layer support were planned from the start, as these capabilities are essential for efficient level editing. The decision to include background music playback within the editor was also made in the planning phase, with the intention of providing a more complete testing environment that simulates how levels would work in the actual game, although it is more of a novelty feature.

In order to facilitate smooth development, it is necessary to have a reliable and straightforward build system. Eventually Make was chosen, accompanied by a Makefile configured to handle compilation and linking tasks. This way, the build process remained simple and platform-independent, allowing easy control over the compilation flags and dependencies without relying on heavier build tools or IDE-specific configurations.

Throughout the planning stage, the focus was on balancing simplicity with functionality. Rather than overcomplicating the tool with niche features, the purpose was to build a stable, fully functional editor that included the essentials needed for 2D level design. This phase helped clarify the scope of the development, set realistic goals, ensure that steady progress could be made and eliminate major structural changes in later stages.

By thoroughly planning the tools, features, and workflow early in the process, a clear vision of well-defined objectives for implementations was achieved. This preparation significantly reduced the risk of scope creep and allowed the development process to remain focused and efficient.

4 TECHNOLOGY STACK

This section provides an overview of the technologies used in the development of the level editor, along with a breakdown of the application's internal structure. Although the development does not follow a typical web-based architecture, the concepts of frontend/UI and backend/core logic still apply when considering the separation of rendering, user interaction, and internal data handling.

4.1 Frontend / UI

The graphical interface and user input handling of the level editor were implemented using Raylib, a lightweight C library designed for game development. Raylib provides the necessary tools to create windows, draw textures, handle mouse and keyboard input, and play audio. In this development,

Raylib was responsible for rendering the grid-based tilemap, tilesets, camera controls, and UI elements such as buttons, toolbars, and selection indicators.

The UI design follows a simple layout that prioritizes usability and clarity. Core actions such as tile placement, selection of tilesets, camera navigation, and layer switching are accessible through keyboard shortcuts and on-screen menus. Basic shape drawing functions from Raylib were used to create interactive elements, for example, buttons and highlighting effects.

4.2 Backend / Core Logic

The backend logic handles all data structures, file operations, and internal state management. This includes the representation of the level grid, tile objects, layers, and the save/load mechanisms. Level data is managed in memory using custom C++ classes and structs, while serialization to disk allows persistent storage of level files.

The backend is also responsible for handling the undo/redo system, keeping track of user actions to enable reversal or reapplication of changes. Additionally, the logic for organizing and categorizing tilesets is maintained separately from the rendering logic, ensuring that the editor remains modular and maintainable.

4.3 Database

No external database system was used in this development. Instead, the editor relies on binary files for level storage. This choice was made to keep the system simple and easily portable, allowing levels to be saved, loaded, and shared without dependency on third-party database systems.

5 IMPLEMENTATION

5.1 Core Features

This section outlines the key features implemented in the level editor, focusing on the essential functionalities required for effective 2D level design. These features were chosen based on common practices found in industry-standard tools and tailored to support a smooth workflow for artists and designers.

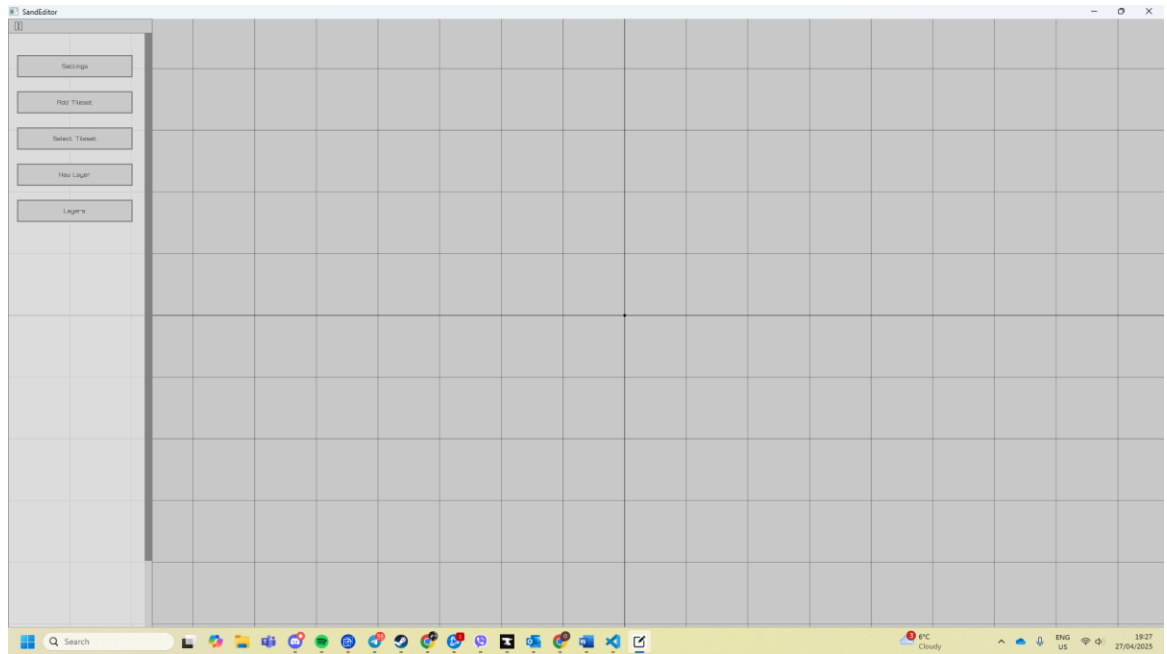


Figure 2. Implementation of the core features at the current stage of the project

5.2 Grid System

At the heart of the level editor is the grid system which provides a structured layout for tile placement (Figure 2). The grid divides the workspace into uniform cells, ensuring consistent alignment of tiles and simplifying the design process. This system not only improves visual organization but also helps maintain logical positioning of game objects within the level. The grid acts as a visual guide and snapping mechanism, reducing errors and enabling precise control over object placement.

5.3 Camera Work

In order to navigate larger level areas efficiently, the editor includes camera controls that allow users to pan and zoom across the workspace. Camera panning is typically handled via keyboard or mouse input, while zooming provides the ability to focus on fine details or gain an overview of the entire level layout. Implementing camera movement is crucial for usability, particularly when dealing with complex or expansive maps where manual scrolling would otherwise be cumbersome.

5.4 Layers

The editor supports multiple layers, allowing different elements of a level—such as background tiles, foreground objects, and interactive elements—to be separated logically. Each layer can be edited independently, providing flexibility in the design process and enabling complex compositions without interfering with other parts of the level. This layering system helps maintain clarity and organization, which is especially valuable when working with overlapping tiles or effects.

5.5 Tile Selection and Placement

Tile selection and placement form the core interactive functionality of the editor. Users can choose tiles from a predefined tileset and place them directly onto the grid (Figure 3). This process supports both single-tile placement and continuous placement through click-and-drag functionality. Accurate tile placement ensures that assets align properly within the level structure, directly influencing the visual quality and gameplay consistency of the final product.

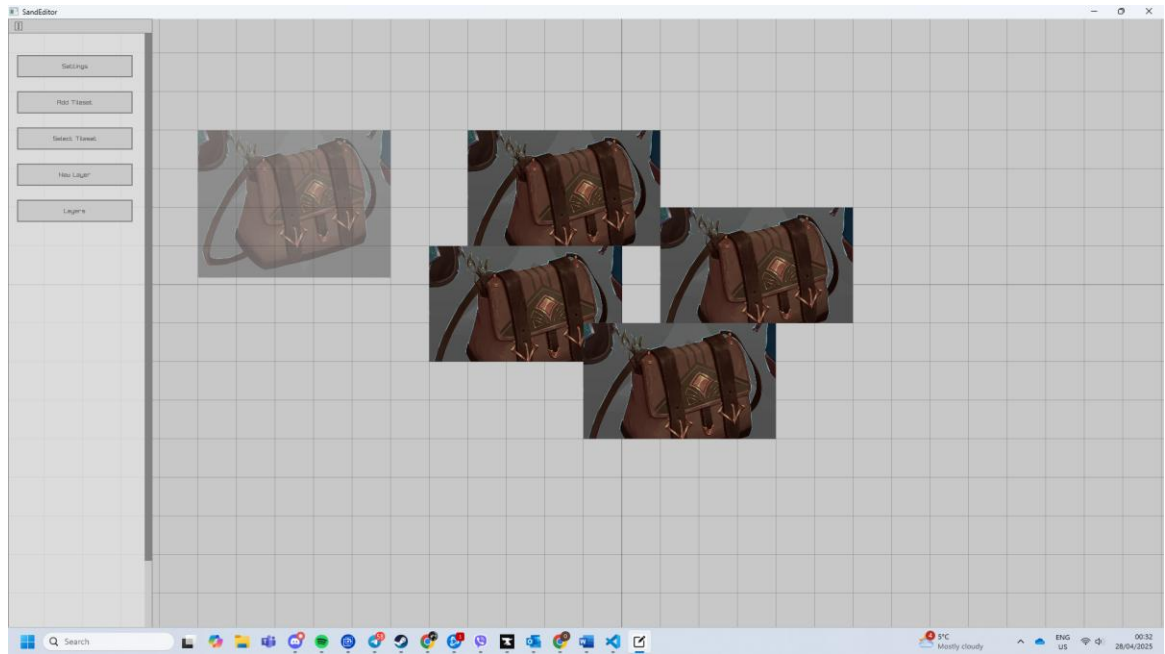


Figure 3. Tile placement and hover effect exemplified

5.6 Select Tool

The select tool (Figure 4) provides functionality for selecting, moving, or deleting tiles within the workspace. By allowing users to click on individual tiles or drag a selection box across multiple tiles, the select tool enhances control over tile manipulation. This feature is particularly useful for rearranging sections of a level or making bulk adjustments without the need to manually delete and replace tiles.

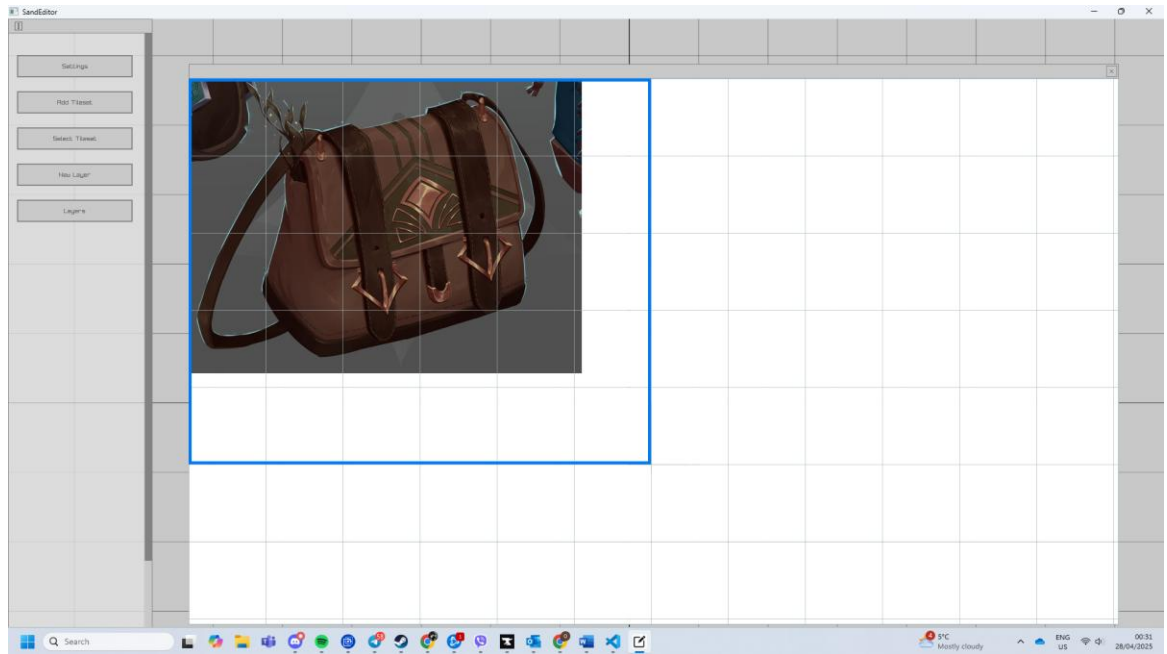


Figure 4. Selection tool

5.7 Tileset Categorization

In order to streamline the process of finding and organizing tiles, the editor implements tileset categorization (Figure 5). Tiles are grouped based on the first three letters of their filename, effectively sorting them into logical categories such as "bg_" for background tiles or "env_" for environmental objects etc. This simple yet effective system helps users quickly navigate large tilesets and improves the overall efficiency of the design workflow.

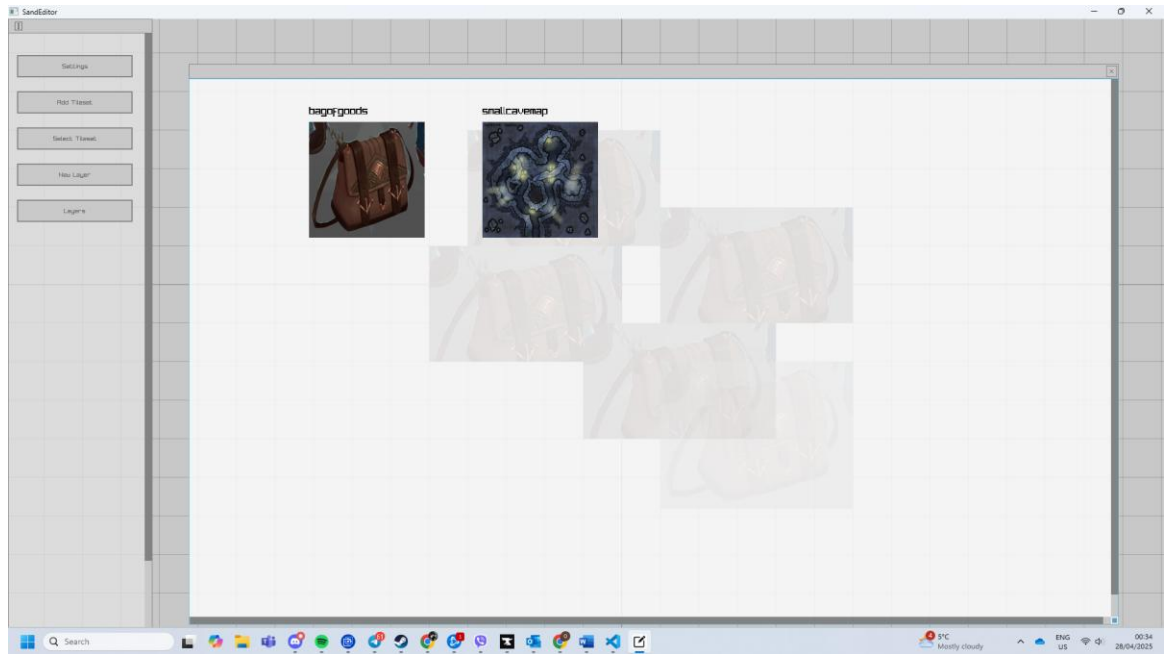


Figure 5. Tileset categorization

5.8 Collision System

The collision system was developed in two versions before the final implementation. At first, all tiles used in the editor were to be categorized in 6 different groups according to their collision. This was subsequently rejected due to the fact that it would bloat the codebase and the tile storage.

Instead, a better solution was to implement the collision directly inside the tile class which would be specified when a tile was added.

5.9 Saving and Loading Levels

A critical feature of the editor is the ability to save and load levels. Levels are stored in a binary format, which captures information about tile positions, layers, tileset usage, and any additional metadata such as background music. This functionality ensures that work can be preserved across sessions and allows for easy sharing or integration of level data into the target game engine. The

save/load system provides the foundation for persistent project management within the editor.

6 CHALLENGES AND SOLUTIONS

6.1 UI Scaling

During development, one of the challenges encountered was ensuring that the editor's user interface scaled properly across different window sizes and resolutions. Without proper scaling, UI elements such as buttons, tiles, and grid lines would appear either too small or too large when the window was resized, negatively impacting usability and visual consistency. In order to address this issue, a straightforward and effective scaling approach was implemented. The solution involved introducing a scale variable, calculated by comparing the current window dimensions to the editor's predefined standard window size. Specifically, the scale factor was derived by dividing the current width and height values of the window by the original design resolution. This ratio provided a dynamic scaling factor that accurately represented how much larger or smaller the current window was relative to the intended baseline.

The calculated scale variable was then applied during the rendering phase of the application, particularly within the update function where all drawing operations occur. By multiplying the positions, sizes, and other relevant parameters of UI elements by this scale factor, the editor maintained consistent proportions regardless of window resizing. This approach ensured that the interface remained both functional and visually coherent across various display configurations without the need for manually adjusting individual element sizes. The solution provided a flexible, maintainable method for UI scaling and contributed significantly to the editor's usability and user experience.

6.2 Serialization

A key requirement for the level editor was the ability to save and load level data reliably and efficiently. Since the tool is designed to support real production workflows, the serialization of level data needed to be both robust and optimized for performance. While text-based formats such as JSON or XML are commonly used for their readability and ease of debugging, they introduce unnecessary overhead in terms of file size and parsing speed, especially as the complexity of levels increases. However, the main concern was that it was still uncertain as to what format the levels would be exported to. For this reason, the decision was made to implement binary serialization for level files.

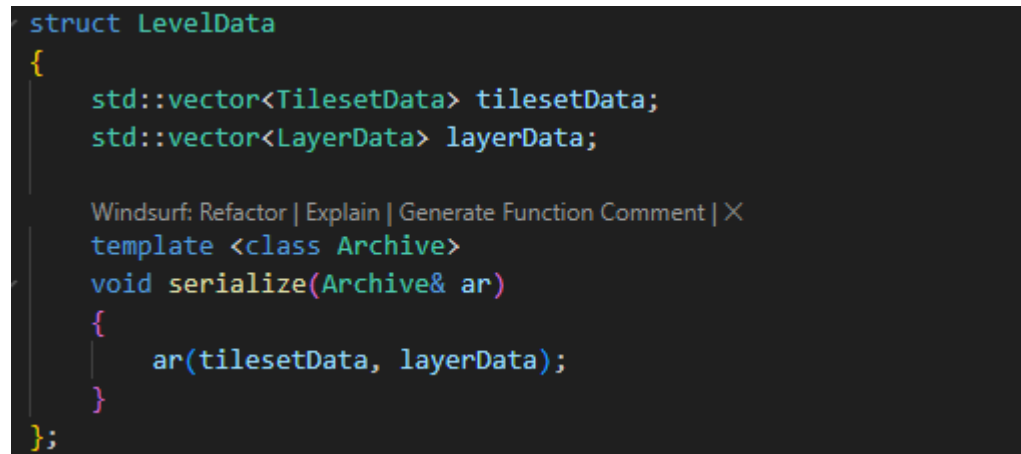
Binary serialization offers several advantages over text-based formats. It produces much smaller file sizes, which makes it easier to store and transfer levels, particularly when working with large maps or complex layer structures. Additionally, binary formats allow for faster read and write operations, reducing load times and improving the responsiveness of the editor when handling file operations. However, manually writing and reading binary data in C++ can be error-prone and time-consuming, requiring careful management of data layouts, alignment, and endianness.

In order to address these challenges and simplify the serialization process, the Cereal serialization library was selected. Cereal is a modern, header-only C++ library that provides an elegant and straightforward way to serialize objects to various formats, including binary, JSON, and XML. Its primary strength lies in its use of C++ templates to enable easy serialization of standard types, user-defined classes, and complex data structures with minimal boilerplate code.

By integrating Cereal into the project, the process of saving and loading level data became highly streamlined. Each relevant data structure, such as the tilemap, layers, and tile properties, could be serialized simply by defining a serialize function within the corresponding class. This function specifies which

member variables should be saved, and Cereal handles the underlying binary output automatically.

For example, a tile object can be serialized as illustrated in Figure 6.



```
struct LevelData
{
    std::vector<TilesetData> tilesetData;
    std::vector<LayerData> layerData;

    Windsurf: Refactor | Explain | Generate Function Comment | X
    template <class Archive>
    void serialize(Archive& ar)
    {
        ar(tilesetData, layerData);
    }
};
```

Figure 6. Serializing a tile object

This approach lowered the complexity of the serialization logic but also improved code maintainability and readability. Should the level data structure evolve in the future, updating the serialization logic will require minimal changes.

The use of the Cereal library, thus, played a crucial role in achieving reliable and efficient binary serialization for the level editor. It allowed for the creation of compact, fast-loading level files while minimizing the risk of data corruption and easing the implementation burden typically associated with manual binary file handling in C++.

7 FUTURE IMPROVEMENTS

There are many potential features that can be added in future iterations of the level editor. One such feature might be the actual export into a game that I am

currently in the making. While the editor is a standalone tool in its current state, the objective from the start was to incorporate it into a developer toolkit.

As for concrete steps forward, an obvious improvement would be to add tilemaps and animation support, similar to those in the Tiled level editor. Features such as basic undo/redo already exist, extending these tools to a full action history with descriptions (e.g., "Placed tile at (5,7)") could enhance usability, especially in further development. An integrated asset manager that helps organize tilesets, music, and sound effects directly inside the editor would also make a welcome addition. This could include previews, categorization, and drag-and-drop placement.

Another item to add could be a simple play-test mode to preview level functionality, including collision and movement simulation to help designers detect issues early before exporting. However, this feature would most likely be implemented when the game for which the editor is made will reach its testing stages.

8 CONCLUSION

Implementing a 2D level editor was not only a technical challenge but also a valuable learning experience in practical game development and modern C++ programming. What began as a focused effort to solve a specific problem—enabling artists to test and place assets without constant programmer assistance—evolved into a deeper exploration of how tools are structured, developed, and maintained in real-world game production workflows.

One of the most impactful aspects of this project was the ability to work extensively with C++. Even for software developers who have previous experience with the language, this project will demonstrate the capabilities of this programming language on a deeper level, introduces an application of more

advanced concepts such as object-oriented design, memory management, modular programming, and file I/O handling. Implementing features like a dynamic grid system, layered rendering, and a flexible tileset loader required structuring the code in a clean, extensible way. Writing serialization routines, integrating third-party libraries like Cereal, and debugging low-level issues gives the reader a more realistic understanding of the kind of problem-solving expected in professional game development environments.

From a game development perspective, building this editor will expose the reader to the internal processes that support a larger game engine or development pipeline.

The development of this level editor has provided a comprehensive exploration into the foundational systems that support larger game engines and development pipelines. By shifting focus from gameplay mechanics and visual assets to the underlying tools that enable content creation, this project offers valuable insights into an often-overlooked yet critical aspect of game development. It highlights the extensive work that occurs behind the scenes—within tooling, automation, and support systems—and emphasizes the importance of developer experience and usability in creating effective software for creative teams.

Through the implementation of key features such as camera movement, undo/redo functionality, and tile categorization, the project translates abstract design objectives into functional and user-friendly systems. The process of planning, scoping, and prioritizing features mirrors the iterative development cycles common in professional software and game production. This approach reinforces best practices in code quality, testing, and maintainability, aligning closely with industry standards.

Moreover, the project bridges technical implementation with creative problem-solving. It showcases how effective tools must not only function correctly, but also be intuitive, efficient, and adaptable to evolving user needs. These qualities can

only be achieved through practical experience in implementation and iteration, rather than theoretical study alone.

Ultimately, this level editor serves not only as a functional product, but also as an educational case study. It deepens understanding of development practices within the game industry, particularly in the areas of tool development, system design, and low-level programming. By revealing the complexity and significance of internal development tools, the project equips readers with a more holistic view of game development and the skills necessary to contribute to such environments meaningfully.

REFERENCES

Cereal. n.d. cereal - A C++11 library for serialization. Web page. Available at: <https://uscilab.github.io/cereal/> [Accessed 19 April 2025].

Godot Engine. n.d. Free and open source 2D and 3D game engine. Web page. Available at: <https://godotengine.org/> [Accessed 19 April 2025].

LÖVE Development Team. n.d. LÖVE - Free 2D Game Engine. Web page. Available at: <https://love2d.org/> [Accessed 19 April 2025].

Microsoft. n.d. Visual Studio Code – Code Editing. Redefined. Web page. Available at: <https://code.visualstudio.com/> [Accessed 19 April 2025].

Photon Storm. n.d. Phaser - A fast, fun and free open source HTML5 game framework. Web page. Available at: <https://phaser.io/> [Accessed 19 April 2025].

Raylib. n.d. A simple and easy-to-use library to enjoy videogames programming. Web page. Available at: <https://www.raylib.com/> [Accessed 19 April 2025].

Tiled. n.d. Tiled Map Editor. Web page. Available at: <https://www.mapeditor.org/> [Accessed 19 April 2025].

Unity Technologies. n.d. Unity Real-Time Development Platform. Web page. Available at: <https://unity.com/> [Accessed 19 April 2025].

FakeFish & Undertow Games. n.d. *Barotrauma*. 2019 Video game. [Accessed 19 April 2025].

LIST OF FIGURES

Figure 1. A screenshot from the game “Barotrauma” which features the level editor (FakeFish & Undertow Games n.d.)	8
Figure 2. Implementation of the core features at the current stage of the project	15
Figure 3. Tile placement and hover effect exemplified	17
Figure 4. Selection tool.....	18
Figure 5. Tileset categorization	19
Figure 6. Serializing a tile object.....	22